Homework — Agentic loop: implementing `BayesFactor`

Purpose

Practice the **agentic coding loop** from the lecture: run a Gemma-powered (`gemma-4-31b-it`) test-fix cycle against a real project, inspect what the model produces, and reflect on how you managed the interaction.

We use Gemma instead of Gemini-2.5 because it is generally less capable and so more sensitive to user error. Also it is freely downloadable and can run on your local machine (in principle).

Background

In The AI-coding landscape we saw this pattern:

```
generate code -> run tests -> read errors -> edit code -> run tests again
```

The example script `files/iterating_regression/iterating_regression.py` implements this as a programmatic loop: you set the right directories, write the prompt file, and run the script. It calls Gemma, writes updated files, runs the tests, and repeats until they pass or `MAX_ATTEMPTS` is reached.

Your task is to adapt that script to implement the `BayesFactor` class from Module 04.

Setup

You need:

- The class Docker container running.*
- A Gemini key in a JSON file or set in the environment.
- Your Module 04 `BayesFactor` test suite.

* Seriously, do *not* let a running agent out of its cage!

Task — `BayesFactor`

Steps:

1. In your existing repo, create a new directory called `week08homework/`. Inside it, create this structure:

   ```
   week08homework/
       agent_loop.py
       task.txt
       bayes_factor.py
       tests/
           test_bayes_factor.py
   ```

   Copy `files/agent_loop/bayes_factor_stub.py` into `week08homework/bayes_factor.py`. Copy your Module 04 test file into `week08homework/tests/`.

   > **Note:** If your test file used the `unittest.expectedFailure` decorator, remove that.

2. Confirm the tests fail before the loop starts:

```
cd week08homework
python -m unittest discover -s tests
```

You should see `NotImplementedError`.

3. Write `task.txt` in `week08homework/`. Your task description must include:
   - The class name (`BayesFactor`) and all required method signatures (`__init__(n, k)`, `likelihood(theta)`, `evidence_slab()`, `evidence_spike()`, `bayes_factor()`).
   - The prior specifications: slab = U(0, 1), spike = U(a, b) with the values of a and b from Etz et al. (2018). (Hint: they are *not* 0.4999 and 0.5001!)
   - The instruction not to modify the test file.
   - The instruction not to change the constructor signature.
4. Before you write `agent_loop.py`, think about the possibility that the model will try to modify the test file. How could it do that? How will you stop it?
5. Write `agent_loop.py` by adapting `files/iterating_regression/iterating_regression.py`. Key parameters to set:
   - `MODEL = "gemma-4-31b-it"`
   - `MAX_ATTEMPTS = <something reasonable>`
   - `TASK_DIR` pointing to `week08homework/`
   - `TEST_DIR` pointing to `week08homework/tests/`
   - The source file set to `bayes_factor.py`

   Run it from `week08homework/`:

   ```
   python agent_loop.py
   ```
6. Review the implementation the model produced. Read through every method before accepting it. How does it smell?

Deliverables

Submit the following in `week08homework/` in your repository:

```
agent_loop.py
task.txt
agent_loop_output.txt   # terminal output of the loop run, copied to a file
bayes_factor.py         # the implementation the model produced (or your edited version)
reflection.md
tests/
   test_bayes_factor.py
```

`reflection.md` must be **at most 250 words** describing how what you did, what happened, did you have to intervene, and what the model got right or wrong.

Do not commit your API key file!